

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

COS214 Project

02/11/2026

GitHub Link

Caleb Jennings	25173805
Anchen Kruger	25073703
Chloe Larsen	25004141
Caitlin Moodley	25128443
Shanna Reinecke	25008260

Task 1: Research Workflow Management Systems

Workflow and WfMS. The Workflow Management Coalition (WfMC) defines a workflow as the computerised automation of a business process, in whole or in part. A WfMS is built around an engine that provides the run-time environment, interprets process definitions, creates instances, and interacts with the people and applications that perform activities [1]. The reference model keeps this engine separate from definition tools, worklist clients, invoked applications, other engines, and monitoring tools, each connected through a defined interface [1].

Definition versus instance. A process definition holds the activities, the rules for moving between them, and the control data. Many instances can be created from it, each with its own data and state [1][2]. The definition owns structure, rules, roles and data declarations. The instance owns current state, data values, assignments and history. Camunda binds each instance to the definition version it started with, so new versions do not alter running work. Migration is a separate step that preserves instance identity and audit trail [5][6].

Work items, nesting and dependencies. Definitions contain related activities and can reference separately defined sub-processes [1][2]. BPMN adds sub-processes and reusable call activities [7]. Ordering relies on a few recurring constructs (sequence, choice, parallel split and join, iteration), and a comparison of 15 commercial products found that none supported all of them [3]. Work therefore needs a hierarchy that supports sequential and parallel composition.

Participants, assignment and routing. The patterns research separates the control-flow, data and resource perspectives. The resource perspective covers who may perform work and how it is distributed [3][4]. Items can be offered or allocated by role, explicit assignment or workload, and can be delegated or escalated [4]. Policies differ between organisations, so they are configured per process, not fixed in the engine.

Approval, rejection, escalation and failure. These appear as ordinary routing outcomes (approve, reject, rework) plus deadlines, escalation to another participant, error handling, and compensation of completed work [4][7]. They are part of the process model, not engine special cases.

Events and external systems. Events can start, interrupt or advance activities. External applications and other engines are reached through interfaces, not embedded in the engine [1][7]. Several parties may need to react to the same occurrence.

History, monitoring and reporting. Monitoring and administration are a separate interface of the reference model [1]. Audit history is tied to instance identity and survives migration [6]. Reporting therefore reads what the engine records instead of living in the core model.

Reusability and configurability. Across [1]–[7], the same responsibilities recur: definition management, instance execution, work distribution, event handling, integration, history and monitoring. The execution mechanism stays fixed. Process content, assignment and routing rules, approval and escalation policy, integrations and reporting vary independently.

Design implications for TIVIDY

1. Separate definition from instance, with per-instance data, state and history.
2. Use a uniformly traversable hierarchy of items, stages and sub-processes.
3. Make assignment, routing, approval and escalation pluggable policies.
4. Give work items state-dependent lifecycles.
5. Keep events, integrations, history and reporting outside the core model, behind narrow interfaces.
6. Bind instances to a definition version and treat migration as an explicit, recorded operation.

References

[1] Hollingsworth, D. (1995). *Workflow Reference Model*. Winchester, Hampshire, UK: Workflow Management Coalition.

[2] The Workflow Management Coalition Specification Workflow Management Coalition Terminology & Glossary. (n.d.). [online] Available at: https://wfmc.org/wp-content/uploads/2022/09/TC-1011_term_glossary_v3.pdf.

[3] van der Aalst, W.M.P., ter Hofstede, A.H.M., Kiepuszewski, B. and Barros, A.P. (2003). Workflow Patterns. *Distributed and Parallel Databases*, 14(1), pp.5–51. doi:10.1023/a:1022883727209.

[4] Russell, N., ter Hofstede, A.H.M., Edmond, D. and van der Aalst, W.M.P. (2005). Workflow Resource Patterns. [online] Queensland University of Technology (QUT) and Eindhoven University of Technology. Available at: www.workflowpatterns.com.

[5] Camunda.io. (2026). *Versioning process definitions | Camunda 8 Docs*. [online] Available at: https://docs.camunda.io/docs/components/best-practices/operations/versioning-process-definitions/.

[6] Camunda 7 community (2026). *Process Instance Migration | docs.camunda.org*. [online] Camunda.org. Available at: https://docs.camunda.org/manual/latest/webapps/cockpit/bpmn/process-instance-migration/.

[7] Object Management Group, *Business Process Model and Notation (BPMN) Version 2.0*, 2011.

Task 2: Define the Scenario and System

Our chosen scenario is employee onboarding across HR, IT, Finance and Training. TIVIDY coordinates the activities needed to prepare a new employee for work. The same engine can support other processes through different work structures, participants and assignment strategies.

Initial scope of TIVIDY

TIVIDY organises a workflow as a tree of work components. An Item represents an individual activity, while an ItemGroup contains items or other groups. Both use the WorkComponent interface so that operations can be applied to individual activities or forwarded through a group.

The WorkflowManager coordinates the root work component, registered workers, assignment strategy, traversal and command execution. Builders support the construction of items and groups, while cloning allows existing work structures to be reused and configured for similar processes.

Main responsibilities of the workflow manager

The initial system will support the following:

- Represent individual work items, stages and nested sub-processes using items and groups.
- Construct work with appropriate names, workers, deadlines, priorities and dependencies.
- Traverse the work hierarchy and filter items by status or assigned worker.
- Keep dependent work blocked until its required approvals have been received.
- Assign work explicitly or select workers using role-based or workload-based assignment strategies.
- Support sequential activities and independent branches through dependencies between work items.
- Enforce state-dependent progression, blocking and cancellation.
- Support review, rejection and escalation through a worker and supervisor hierarchy.
- Execute assignment, progression, rejection, escalation and cancellation through commands, retaining an in-memory command history.
- Allow events to be published to dependency, notification and audit observers.
- Support email notification through a common interface and an adapter to a legacy email service.

System boundaries

TIVIDY controls work organisation, availability, assignment, progression and tracking. The organisation determines the required activities, responsibilities and review levels. Workers perform the activities, make review decisions and handle exceptions.

Departmental systems remain responsible for their specialised operations. For example, IT creates accounts, Finance registers employees for payroll, and HR maintains employee records. The responsible workers confirm the results of those activities in TIVIDY.

Workflow information

Element	Information and responsibility
WorkflowManager	Coordinates the root work component, creation time, registered workers, assignment strategy and command history.
Item	Represents an individual activity with a name, deadline, priority, assigned worker, current state, outstanding dependency count and rejection information.
ItemGroup	Organises child work components, which may be individual items or further groups.
Worker	Represents a participant with an identity, name, role, supervisor, subordinates and an escalation queue.
Event	Identifies a workflow occurrence through its type, source item and timestamp.

The item states are Blocked, Available, InProgress, Approved, Completed and Cancelled. Assignment is recorded separately from the lifecycle state. Rejection records a flag and reason, while escalation is tracked separately and can transfer responsibility to a supervisor.

Parts of the system expected to vary

Different processes can use different work hierarchies, dependencies, participants, deadlines and priorities. Assignment can vary between role-based and workload-based strategies. Traversal can use dependency or priority iterators, with filtering by status or assignee.

Notification, dependency and audit observers can be attached where required. The email service implementation can also change behind its common interface.

Employee Onboarding Scenario

Organisation and environment

The scenario takes place in a medium-sized organisation with HR, IT, Finance and Training departments. After an employee accepts an offer, HR verifies their information, obtains approval for the onboarding plan and coordinates departmental preparation.

Process being managed

The process coordinates the preparation required for a newly hired employee to join the organisation. It begins when HR creates the employee's onboarding work tree. It ends successfully when the required preparation, reviews and employee-record update have been confirmed. If onboarding must stop, HR cancels the remaining active work.

The employee's department, job role, location, start date and resource requirements guide the selection and configuration of onboarding activities.

Participants and responsibilities

Participant or role	Responsibility
New employee	Submit requested information and documents, correct missing or incorrect details, and attend orientation.
HR officer	Create and organise onboarding work, verify information, coordinate corrections and perform the final readiness review.
Line manager	Review the onboarding plan, confirm required access and resources, and handle reviews or escalations assigned to their level.
Director	Handle reviews and escalations passed to their level through the supervisor hierarchy.
IT staff	Prepare accounts, role-appropriate access and equipment, and confirm the results.
Finance or payroll staff	Verify payroll information, register the employee in the departmental system and confirm completion.
Training coordinator	Arrange the initial orientation briefing and record its completion.

Major pieces of work

1. Create onboarding work

HR creates the onboarding group and its required activities. Builders configure item names, workers, deadlines, priorities and observers. Similar existing items or groups may be cloned and adjusted for the employee.

2. Verify employee information

The employee supplies the requested information and documents. HR checks that the information is complete and correct.

Missing or incorrect information leads to a recorded rejection reason and a request for correction. HR coordinates another review after the employee provides the corrected information. Dependent work remains blocked until the required approval has been obtained.

3. Review the onboarding plan

The onboarding plan identifies the required access, equipment and other preparation. Review requests pass through the worker and supervisor hierarchy for the required number of approval levels.

Each required reviewer makes a decision. A rejection stops the review chain and records a reason. HR then coordinates corrections or decides whether onboarding should be cancelled. Requests that cannot be resolved through the available hierarchy require escalation and follow-up.

4. Carry out departmental preparation

Once the relevant approval dependencies have been satisfied, departmental preparation can proceed through independent branches:

- IT preparation: Create the required account, configure access and prepare equipment. Account approval is a prerequisite for the dependent access activity.
- Payroll preparation: Verify payroll information, register the employee in the departmental system and confirm the result.
- Orientation: Schedule and complete an initial briefing that can take place independently of account preparation.

HR includes any additional activities required for the employee. For example, remote employees may require remote-access preparation, while on-site employees may require site-access preparation.

5. Check final readiness

HR's final readiness item depends on approval of the required departmental preparation items. Approval of the IT branch alone is insufficient; payroll, orientation and any additional required work must also satisfy their approval dependencies.

HR checks the departmental results and confirms that the required activities have reached their completion stage. Problems are returned to the responsible department for correction and another review.

6. Confirm the employee-record update

After the readiness review is accepted, HR updates the employee's organisational record outside TIVIDY and confirms the corresponding final task.

Onboarding is considered successful when all required preparation, reviews and final activities have been confirmed.

Important decisions and dependencies

HR selects the required tasks when constructing the onboarding tree. The employee's circumstances determine which additional activities are included. Only the selected activities become prerequisites for later work. Information verification precedes approval of the onboarding plan. Plan approval precedes departmental preparation. Within IT preparation, account approval precedes the dependent access activity.

The final readiness item depends on the required departmental approvals. Independent branches can remain active at the same time. When ItemGroup forwards a progress operation, it calls its children in order.

Dependencies determine whether work can proceed, while the assignment strategy selects the worker responsible for it.

Reviews and escalation

Workers are linked to supervisors. A review request can require approval through several levels of this hierarchy. Each required reviewer must approve the request for the chain to succeed. An explicit rejection stops the chain and records its reason.

If the required review cannot be completed through the available hierarchy, the request requires escalation and follow-up. A supervisor handling an escalation can take responsibility for the item and place it in their escalation queue. Escalation and Rejection are recorded separately from the item's lifecycle state. HR decides whether the work should be corrected or the remaining onboarding activities cancelled.

Deadlines record target dates. HR or another responsible participant identifies overdue work and arranges reassignment or escalation where appropriate.

Failures and exception routes

Condition	Workflow response
Missing or invalid information	Record the reason, request correction and arrange another review before dependent work proceeds.
Approval requires revision	Return the work for correction and submit it for review again.
Approval is finally rejected	Retain the rejection reason. HR decides whether to stop onboarding and cancel the remaining active work.
Required review level is unavailable	Escalate the item for follow-up through the worker hierarchy.
Assignee is unavailable or a deadline has passed	Arrange reassignment or escalation as appropriate.
Departmental preparation fails or is incomplete	Correct the departmental problem and obtain the required approval before downstream work proceeds.
Required external update fails	Keep the corresponding activity unfinished while the responsible department resolves the problem and confirms the update.
Notification problem occurs	Investigate the email service separately and keep the work-item outcome based on the actual work performed.
Employee withdraws or HR cancels	Cancel remaining active work where permitted by its state and arrange departmental cleanup of resources already created.

External systems

The Notification observer uses the EmailService interface to produce assignment, completion, approval, rejection and escalation notices. EmailServiceAdapter translates the subject and body into the HTML format accepted by LegacyEmailService. The current legacy service demonstrates delivery by writing the message to console output.

Expected outcome

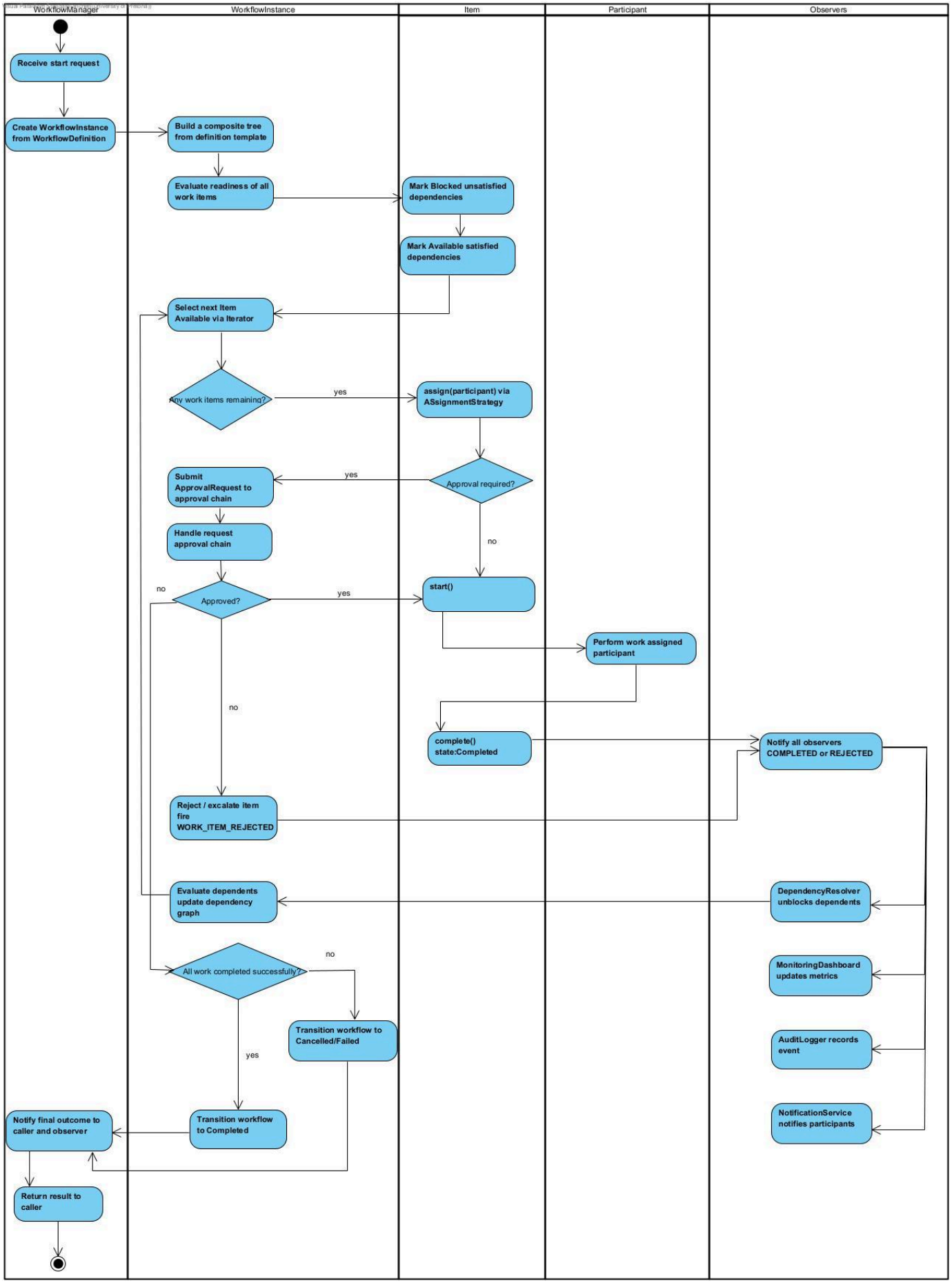
Successful onboarding leaves the employee’s information accepted, the onboarding plan reviewed and approved, IT preparation confirmed, payroll registration confirmed, orientation completed, and the final readiness review and employee-record update finished.

The work tree shows the activities and their outcomes. Assigned workers and rejection reasons provide context for follow-up. Commands executed through WorkflowManager remain in its in-memory history, while published events can produce timestamped audit output.

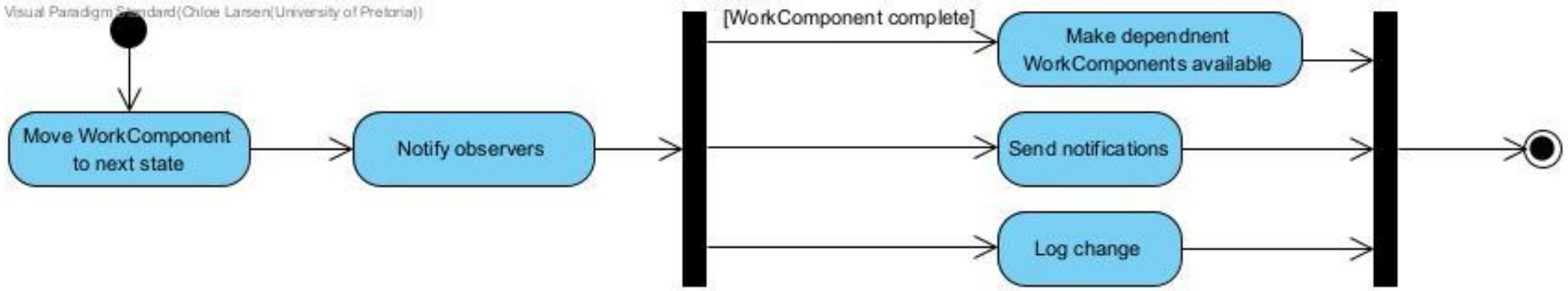
If onboarding stops, the remaining active work is cancelled where permitted by its state. Departments remain responsible for cleaning up any accounts, equipment or registrations already created.

Task 3: UML Activity Diagrams

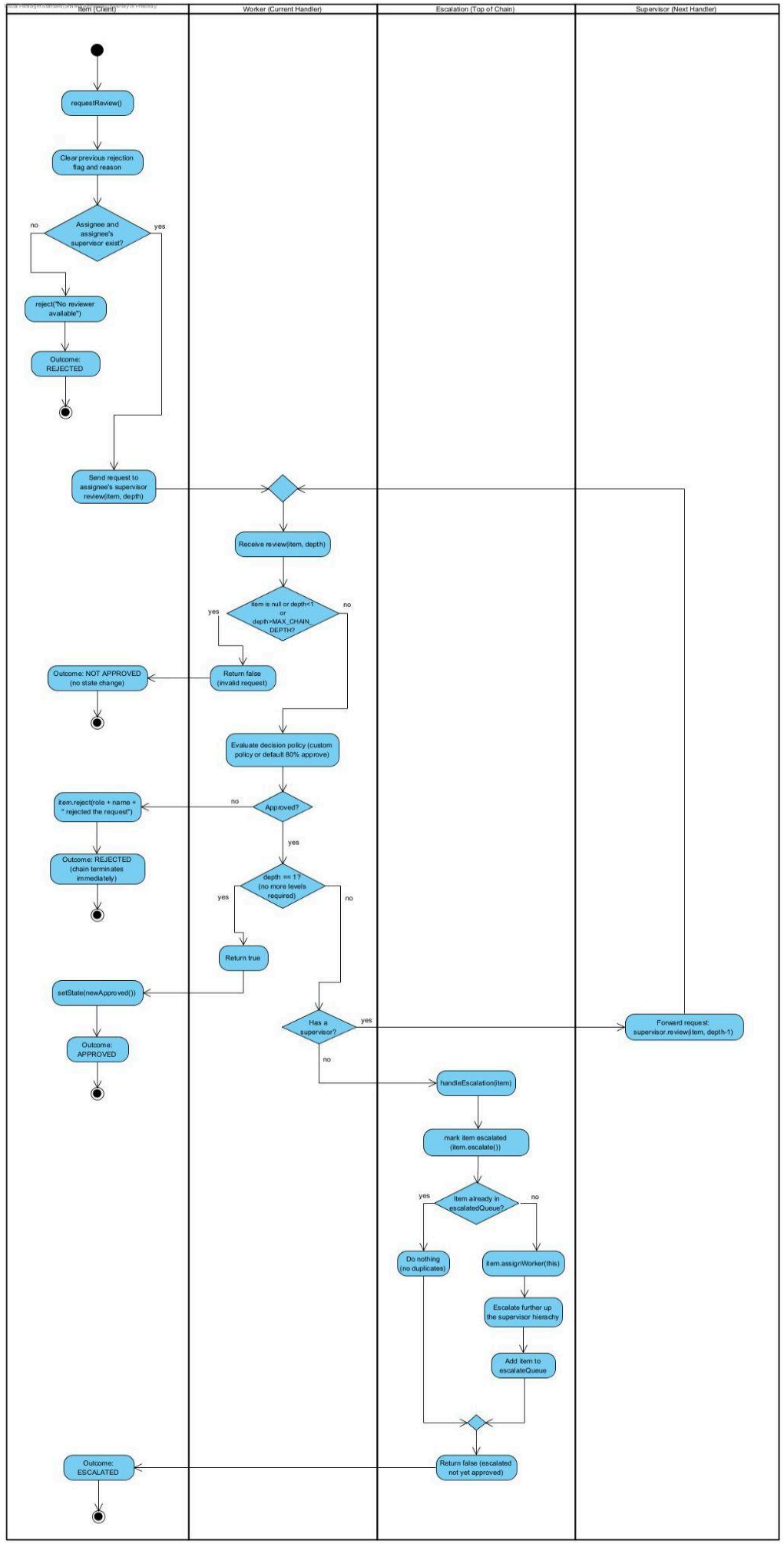
Activity Diagram 1



Activity Diagram 2: Handling Component Events



Activity Diagram 3



Task 4: Design Patterns

Composite

Component	WorkComponent
Leaf	Item
Composite	ItemGroup
Client	WorkflowManager

Design Problem

Work that needs to be completed is hierarchical, a workflow instance contains stages, which can contain work items or sub-processes. The rest of the system will treat individual items and groups of items uniformly without knowing the internal structure of the item. Hence the use of the composite pattern.

Collaboration

WorkflowManager holds a reference to the root of a *WorkComponent* tree. Therefore the *process()* operation can be called on the root and can then propagate down the tree recursively. *ItemGroup* delegates the result of the *process()* operation, while Item performs the actual work that needs to be performed.

What Varies

WorkComponent items can be added to the created tree without modifying the existing clients. Therefore following the open-closed principle.

Why Appropriate

The specification indicated that work is not flat and contains stages, individual work items, and sub-processes, the use of the composite pattern helps to implement the hierarchical structure.

Iterator

Iterator	WorkflowIterator
ConcreteIterator	DependencyIterator, PriorityIterator, Filtered
Aggregate	WorkComponent
ConcreteAggregate	ItemGroup, Item

Design Problem

The system will need to traverse running workflow structures without exposing the internal representation of the composite structure.

Collaboration

WorkComponent provides *createIterator()* method. The concrete aggregates return an appropriate iterator that can be used by the clients. The client can use the *first()*, *next()*, *isDone()*, and *currentItem()* methods to iterate through the structure without knowing what the internal iteration structure is.

What Varies

The iterator offers depth first traversal, breadth first traversal and filtered traversal.

Why Appropriate

The specification indicated that the hierarchical structure of the composite pattern must be iterable. The iterator pattern decouples the traversal algorithm from the hierarchical structure allowing multiple different traversal strategies to be implemented.

State

Context	Item
State	State
ConcreteState	Available, Approved, InProgress, Completed, Cancelled, Blocked

Design Problem

The use of the state design pattern helps to represent the complex lifecycle of an item. An item can be created, made available, assigned, in progress, completed, rejected, cancelled, or escalated. The use of the state pattern helps to prevent the large if/switch statements.

Collaboration

Item delegates the lifecycle operations (*progress()*, *cancel()*, *block()*, *unblock()*, and *cancel()*) to the current *State* object. Each ConcreteState will decide if the chosen lifecycle operation is valid and what the next state shall be. State transitions are localised to the state classes.

What Varies

The valid transitions and behavior associated with each lifecycle phase

Why Appropriate

The specification specifies that the item-lifecycle which is correctly represented and implemented by the State pattern. The use of the State pattern helps to eliminate complex conditional checks.

Strategy

Contect	ManagerWorkflow
Strategy	AssignmentStrategy
ConcreteStrategy	RoleBased, WorkloadBased

Design Problem

Different departments can assign work based on different methods for assigning work. One could use Role Based methods, and another could use Workload Based methods

Collaboration

WorkflowManager holds an *AssignmentStrategy* object and it delegates the *assign()* call to its children.

What Varies

The algorithm that decided who completed the next task and the algorithm that decided on what work comes next.

Why Appropriate

The specification specifies that assignment and routing strategy must be configurable and the use of the strategy pattern helps to enforce this.

Factory Method

Creator	ComponentBuilder
ConcreteCreator	ItemGroupBuilder, ItemBuilder
Product	WorkComponent
ConcreteProduct	ItemGroup, Item
factoryMethod()	createProduct()

Design Problem

We need a way to dynamically construct different types of items (composite vs leaf) - i.e. same methods on different polymorphic instances. We do this with a factory method (for creating the concrete type) and using the instance it returns as the object that is built upon in a simple builder that supports method chaining for easy construction.

Collaboration

Used semi-directly by the client to create new workflow items.

Prototype

Prototype	WorkComponent
ConcretePrototype	ItemGroup, Item
Client	WorkflowManager

Design Problem

If a WorkComponent object needs to be cloned the prototype pattern can ensure that a deep copy clone is created and returned correctly so that if the Cloned object is a part of the *ItemGroup* class then all of its children are correctly cloned as well.

Collaboration

The system or user triggers workflow creation via WorkflowManager, which allocates a new WorkflowInstance and invokes clone() on its blueprint root template pointer.

What Varies

The object cloned can vary depending on the object that the clone is called upon

Why Appropriate

The Builder pattern constructs the intricate blueprint tree once during system setup. Prototype then enables the engine to create hundreds of active runtime executions cheaply and safely from that single blueprint without re-running the multi-step builder logic every time an instance starts.

Command

Command	WorkflowCommand
ConcreteCommand	Assign, Progress, Cancel, Reject, Escalate
Receiver	Item
Invoker	WorkflowManager

Design Problem

Actions performed on items (Assign, Start, Complete, Reject, Escalate) need to be encapsulated as objects to support rollback/undo commands. The encapsulation of the object also ensures there is decoupling of the invoker to the receiver.

Collaboration

Each action is represented by a *WorkflowCommand* object. *WorkflowManager* executes the command and stores them in its history and rollback attributes. Command delegates the actual to *Item* which is the receiver.

What Varies

The set of operations that can be performed, undone or redone.

Why Appropriate

The specifications require that completed actions and state changes must be stored and rollback must be implemented. So the use of the Command pattern helps to ensure that the specification is followed.

Observer

Subject	EventSource
Concrete Subject	WorkComponent
Observer	Observer
ConcreteObserver	Dependency, Notification, AuditLogger

Design Problem

The completion of one task may impact the availability of other tasks, an external event could trigger new tasks, or several interested parts of the system may be interested in the results of a task. This indicates that the observer pattern must be used.

Collaboration

When an *Item* changes its state, it will notify all attached observers by calling the *update()* with the *Event*. Each observer will then react independently: *Dependency* will unlock the next task, *Notification* will send emails and *Audit Logger* will record the change.

What Varies

Depending on who is attached to events the reaction to the events shall differ. New observers can be added without modifying the existing observers.

Why Appropriate

The specification requires that several interested parts of the system react to the same event in different ways depending on the chosen observer type. The observer pattern helps to implement.

Adapter

Target	EmailService
Adapter	EmailServiceAdapter
Adaptee	EmailService
Client	Notification

Design Problem

Allow the TIVIDY system to communicate with external systems like email

Collaboration

The client class calls the *sendEmail()* method defined in the *ExternalSystemInterface*. Each adapter will translate the request into the required form by the adaptee.

What Varies

Depending on the instance of the EmailService that is called will dictate what the response will be. New adapters can be added without changing the core workflow model.

Why Appropriate

The specification requires that communication with external systems must take place and the TIVIDY system must not be implemented to contain these interfaces already. The adapter pattern will help implement this.

Chain of Responsibility

Handler	Worker
ConcreteHandler	Worker

Client	Item

Design Problem

The approval of an item can involve multiple people in a successor hierarchy. An approval request may have to move up multiple levels of workers.

Collaboration

An item is sent to its worker’s superior for approval. That superior then chooses to approve or reject. If it rejects, it terminates the chain. Otherwise, if the priority is high enough, it recursively forwards the request to its superior for further approval. The higher the priority (importance), the further along the chain the request goes.

What Varies

The number of approval levels and who approved the request.

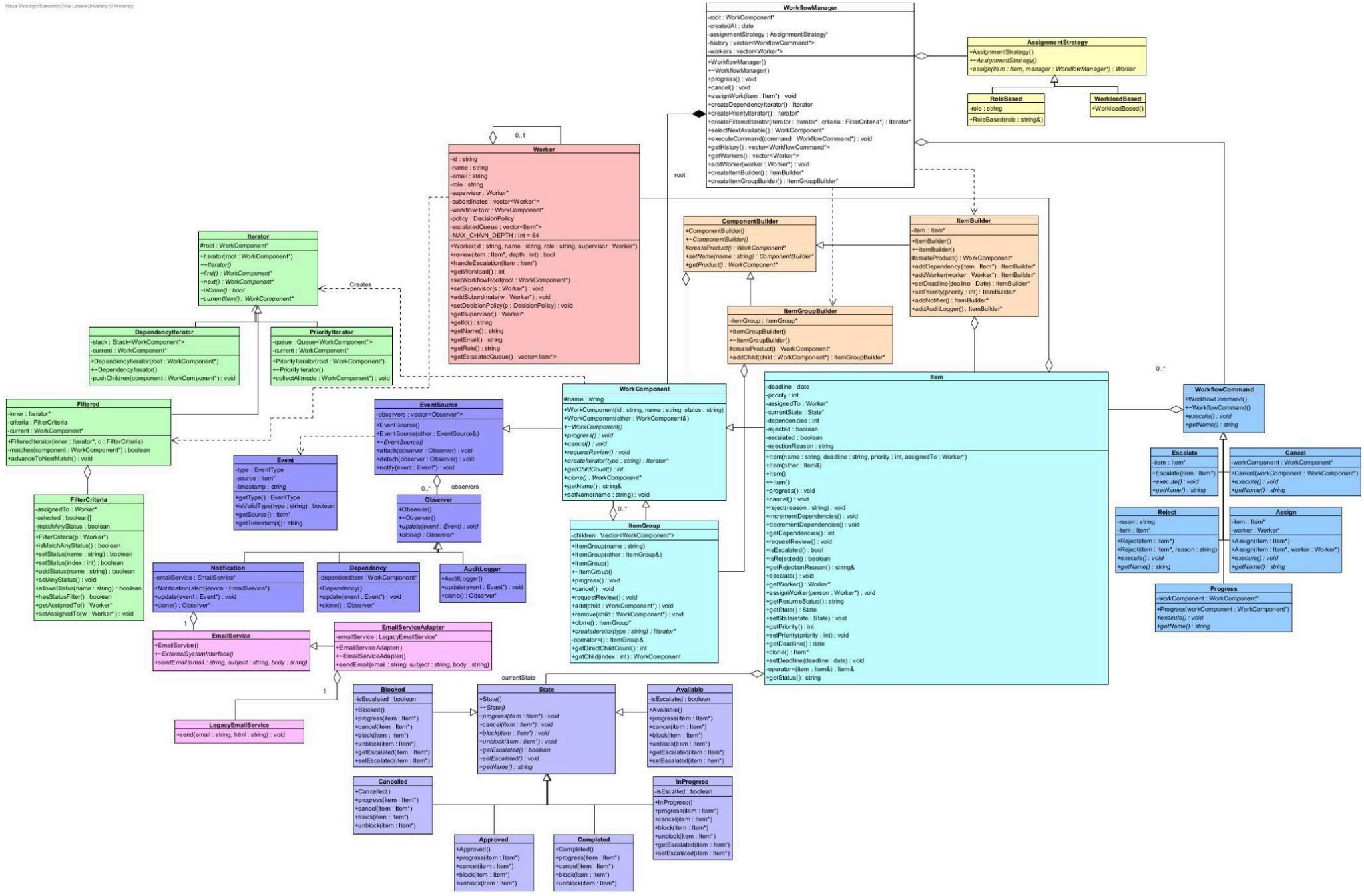
Why Appropriate

The specifications required that there should be multiple levels of approval and escalation. The Chain of Responsibility pattern provides a structure where this is possible.

Note

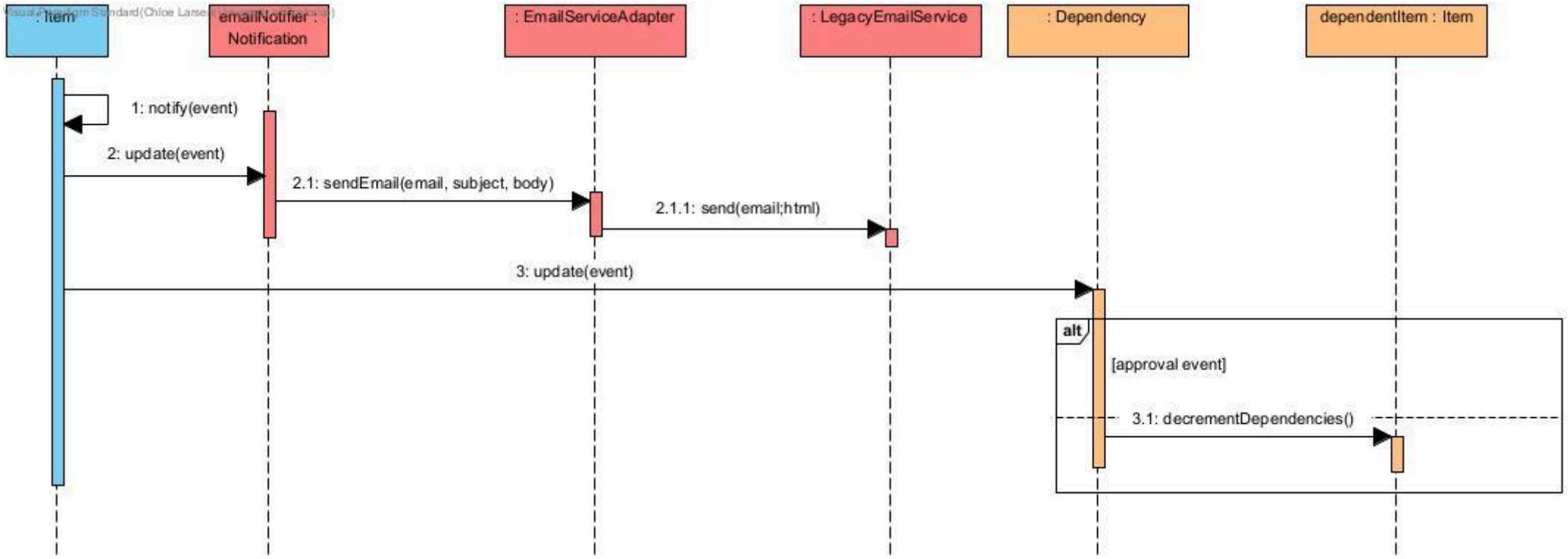
The abstract and concrete handler are the same class because only a single concrete handler is needed in this system, and so having an abstract parent class would be redundant.

Task 5: UML Class Diagram

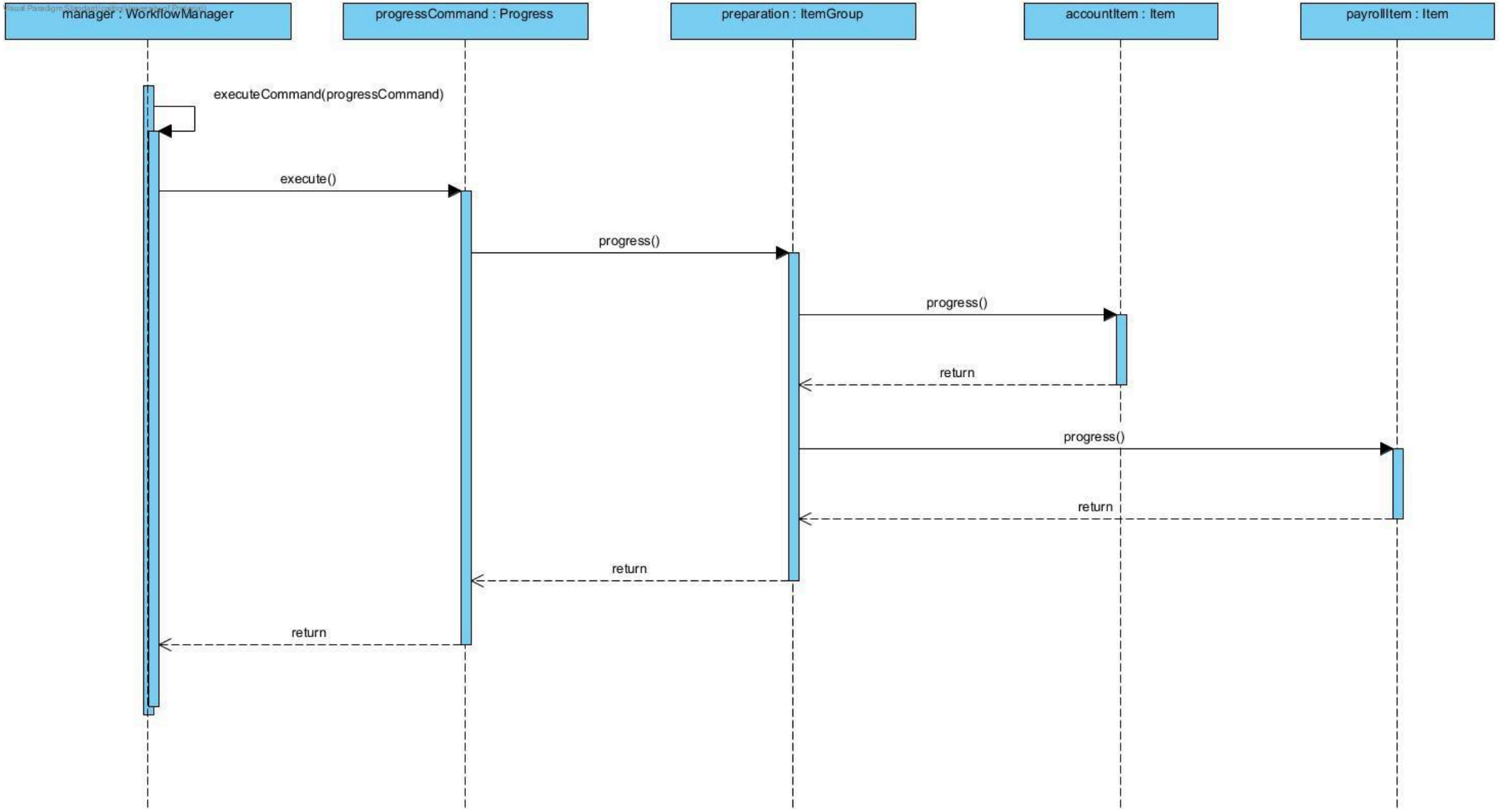


Task 6: Runtime Behaviour Diagrams

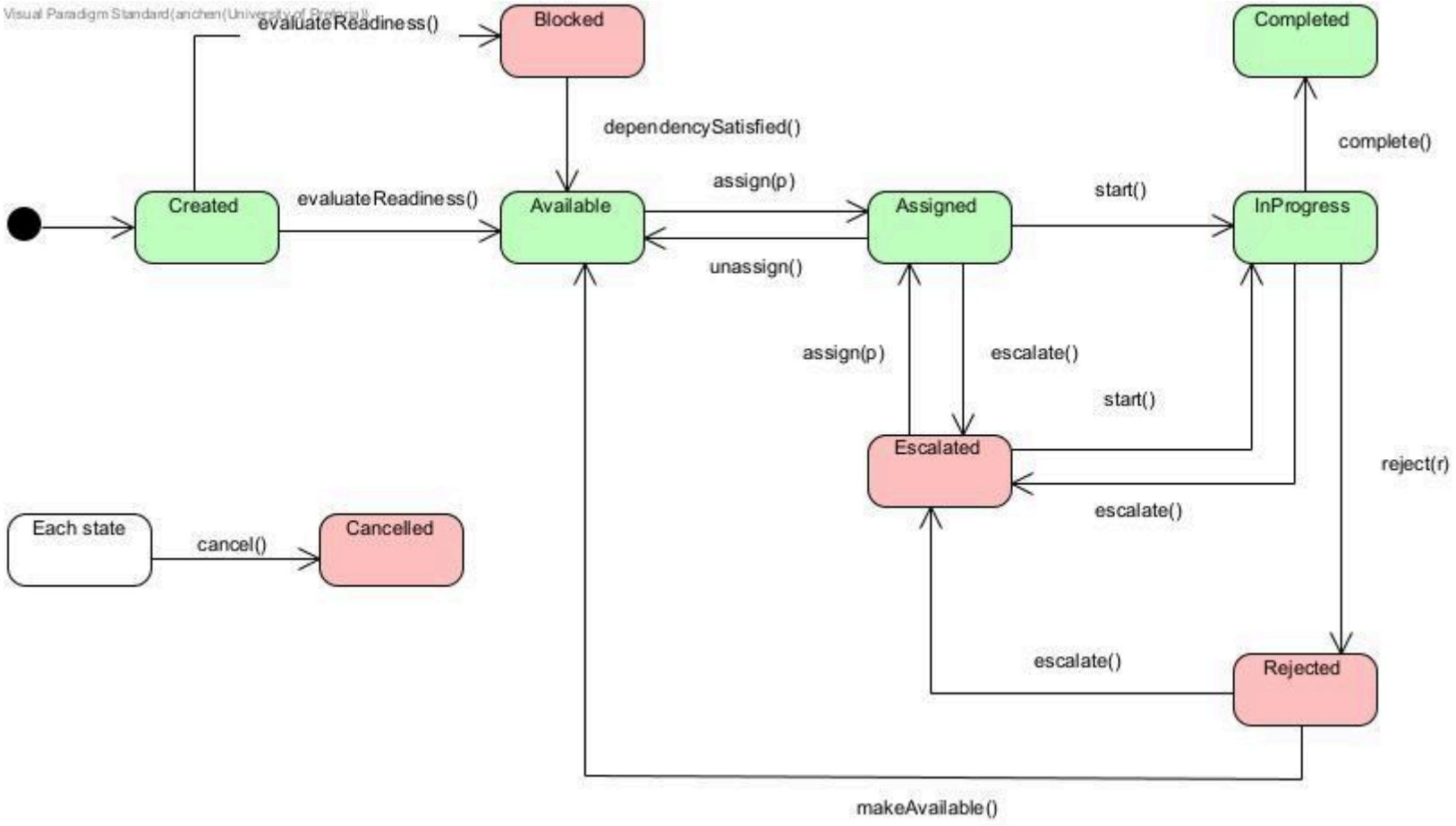
Sequence Diagram 1: Linking to External Services



Sequence Diagram 2 : Progress Workflow Items



State Diagram

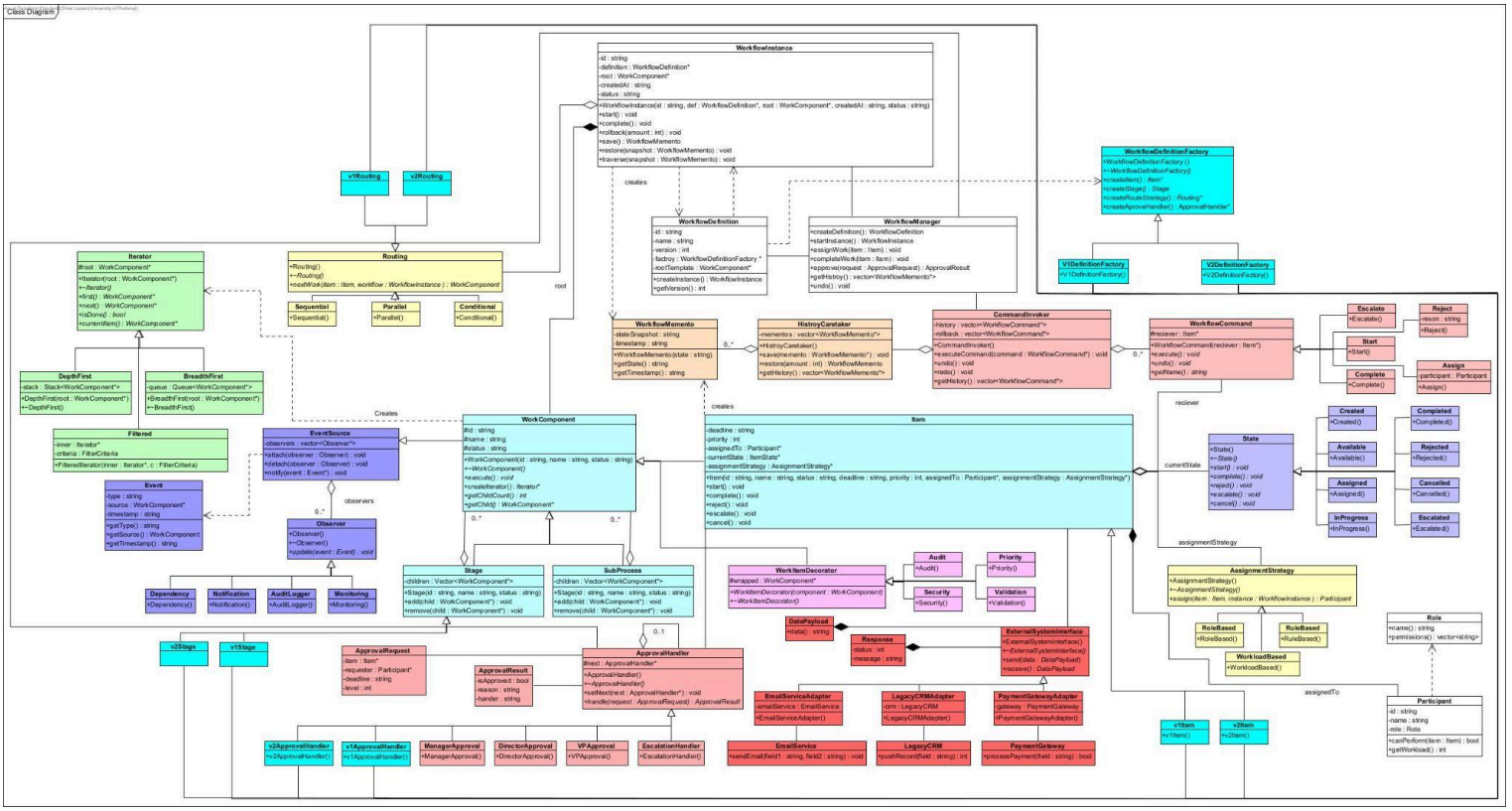


Task 7: Design Decisions and Revisions

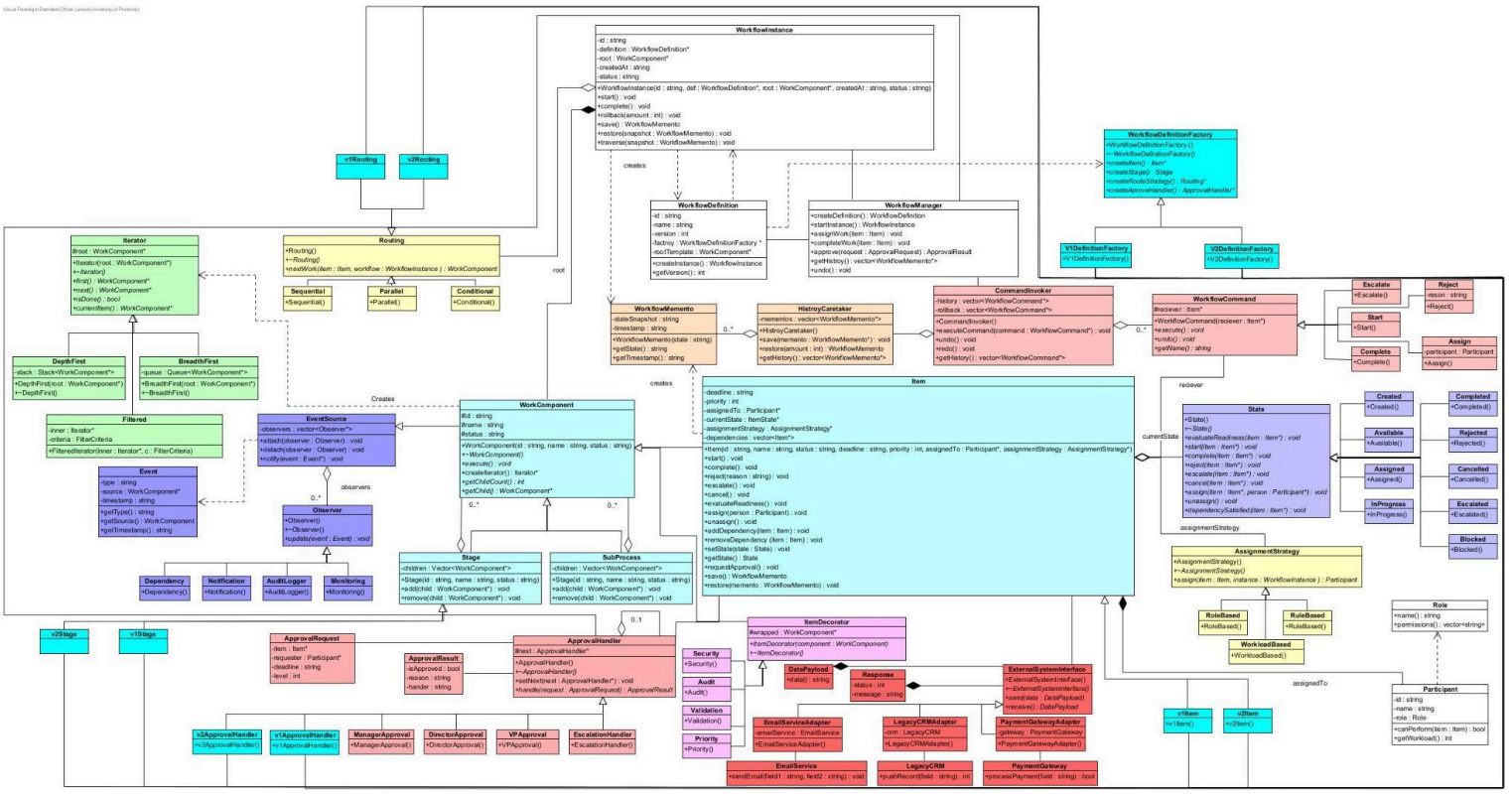
Revisions:

Class Diagram:

Version 1:



Version 2:



Revision 1 - Adding *Blocked* state:

The problem or question being considered

In the original implementation of the design lacked the implementation to have work items be in a blocked state when they cannot be completed due to dependencies.

The original design or assumption

The original design did not take the potential blocking of an item.

The change that was made

The *Blocked* state was added to the state pattern and retroactively the functions *evaulateReadiness()* and *dependencySatisfied()* were added to the *State* and *Item* class.

Why the change was made

The changes made provided the interface for the *Blocked* state to be implemented.

Updates to the relevant diagrams

Blocked class was added as a child of the *State* class. To the *State* class the functions *dependencySatisfied()* and *evaluateReadiness()* were added. To the *Item* class the function *evaluateReadiness()* was added.

Revision 2 - Adding missing functions to the *Item* class:

The problem or question being considered

WIth the original design there was a gap in the functions declared and how they link to state. So we added more functions to the *Item* and *State* classes to provide declared functionality.

The original design or assumption

There was no accessor or mutator for *currentState* in *Item*. There was no function to assign a person to the item nor was there a function to unassign a person. There was no function that would start the chain of responsibility. No functions to save to the memento or rollback.

The change that was made

An accessor and a mutator was added for *currentState*. A function for assigning a task has been created, and another function for unassigning was added. A function to use the chain of command was added. A save and rollback function for memento was added.

Why the change was made

All of these design patterns were added and have been dictated to interact with *Item* and the original implementation did not contain the functions in order to fulfill the requirements.

Updates to the relevant diagrams

getState() and *setState()* was added to *Item*.
assign(person : Participant) and *unassign()* was added to *Item*.
save() and *restore(memento: WorkflowMemento)* was added to *Item*.

Revision 3 - Adding missing functions to the *State* class:

The problem or question being considered

In the *State* class there was no functions implemented for the Assigned state

The original design or assumption

No functions were added in the original design.

The change that was made

A function for assigning a person was added to *State* and another function was added to unassign the assigned person.

Why the change was made

Implementing the two functions ensured that the Assigned state could be used correctly.

Updates to the relevant diagrams

assign(item : Item, person : Participant*)* was added to *State*.
unassign() was added to *State*.

Revision 4 - Adding dependency to *Item*

The problem or question being considered

In the *Item* class we thought that we forgot to implement noting that an item can have dependencies.

The original design or assumption

The original design decided that dependencies were noted by the dependency class in the observer pattern.

The change that was made

In the *Item* class there is a vector of *Item* objects to store all of an object dependencies and an appropriate accessor and mutator was added.

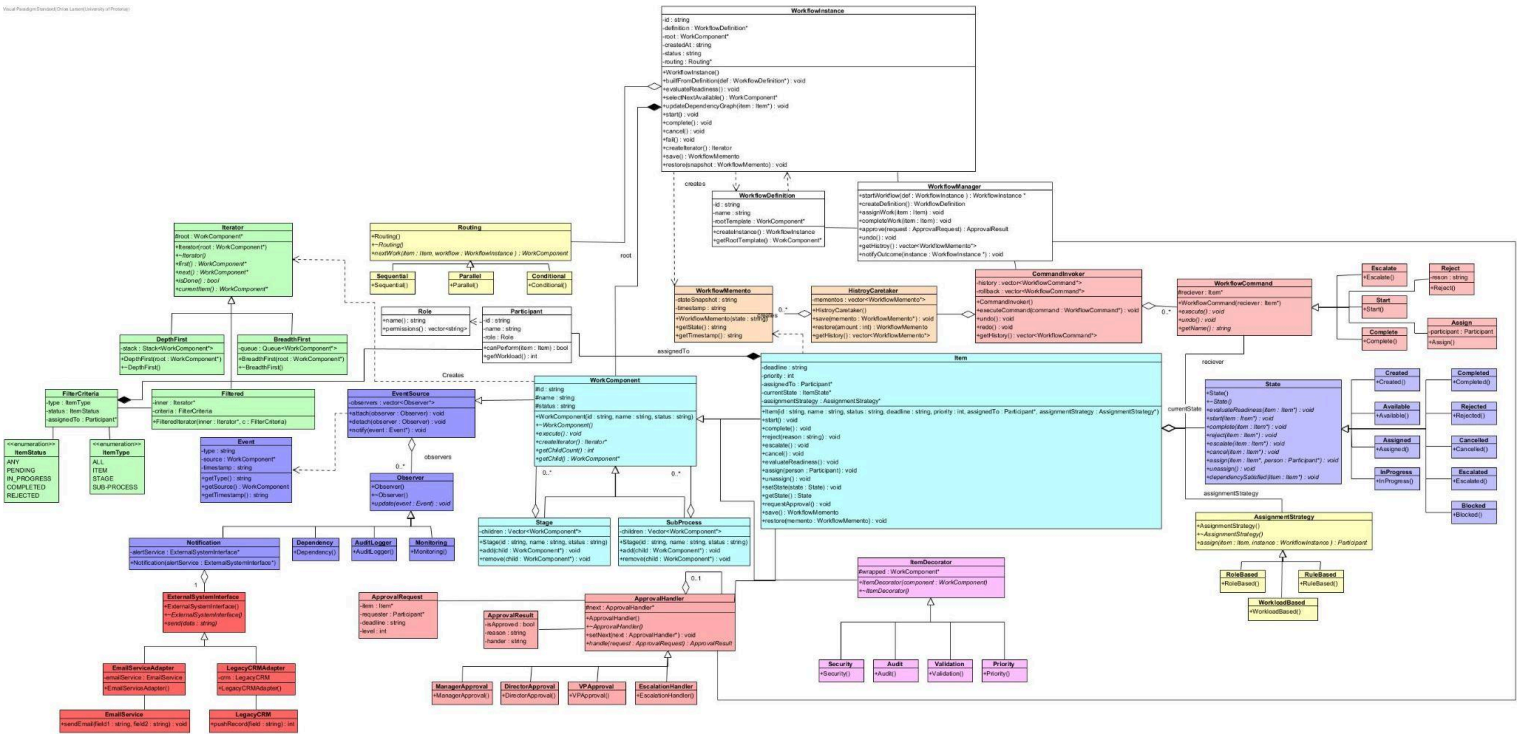
Why the change was made

An item cannot progress state wise if the item has dependencies. Therefore the implementation of the dependencies will help to ensure that an item can progress.

Updates to the relevant diagrams

*vector<Item *> dependencies* was added to *Item*.
addDependency(item: Item)* was added to *Item*.
removeDependency(item : Item)* was added to *Item*.

Version 3:



Revision 1 - Removal of the Abstract Factory pattern:

The problem or question being considered

We did not need to implement versioning like we already assumed, all of the implementation we have previously completed already satisfies all requirements specified in the specification

The original design or assumption

When we originally designed the system to include versioning and the best way we thought to implement it was to use the abstract factory design pattern.

The change that was made

The abstract factory classes were removed.

Why the change was made

The removal of the classes help to remove the implementation of the abstract factory pattern.

Updates to the relevant diagrams

Removal of *WorkflowDefinitionFactory*, *V1DefinitionFactory*, *V2DefinitionFactory*, *V1Item*, *V2Item*, *V1ApprovalHandler*, *V2ApprovalHandler*, *V1Stage*, *V2Stage*, *V1Routing*, *V2Routing*.
version: int & *factory:WorkflowDefinitionFactory* in *WorkflowDefinition* was removed.

Revision 2 -Removing dependencies from *Item*:

The problem or question being considered

In the previous version of the diagram dependency functionality was added to the *Item* class whereas it was actually implemented in the observer pattern.

The original design or assumption

An item cannot progress state wise if the item has dependencies. Therefore the implementation of the dependencies will help to ensure that an item can progress.

The change that was made

Removal of dependencies vector and *addDependency(item: Item*)* and *removeDependency(item : Item*)* was removed

Why the change was made

The dependency observer completes the same work as the vector. So the use of the observer will be a better implementation of dependencies.

Updates to the relevant diagrams

I Removal of *vector<Item *> dependencies*, *addDependency(item: Item*)*, *removeDependency(item : Item*)* from *Item*.

Revision 3 - Moving Adapter pattern to be used by the Notification Observer

The problem or question being considered

Each individual item was linked to the external system interface which would have been ok but there is already a Notification observer so it would have been better to use this for notifications for external systems.

The original design or assumption

I do not know

The change that was made

The adapter pattern was moved to be included in the Notification Observer. With a pointer in the notification observer.

Why the change was made

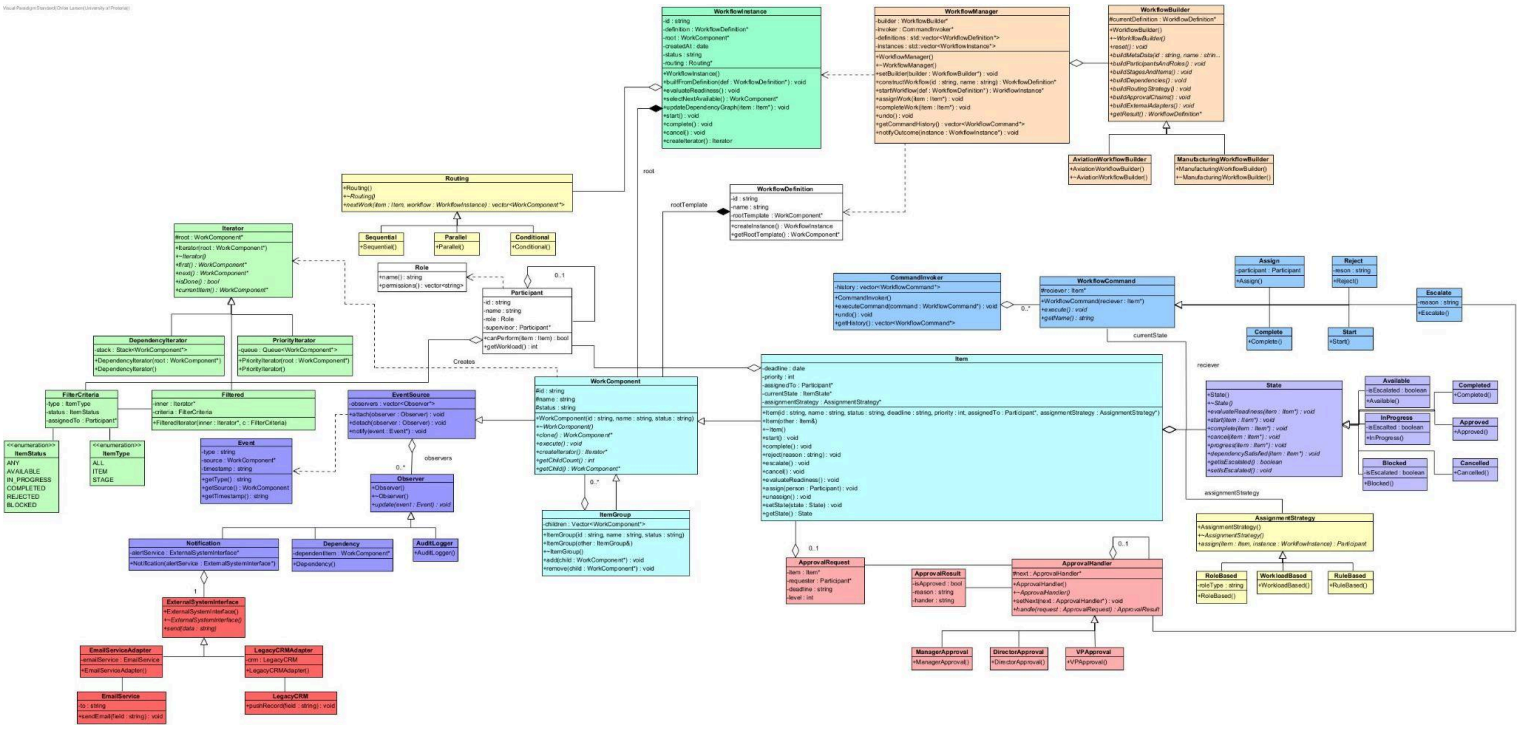
The linking of the notification observer and the adapter will help complete notifications in an easier matter

Updates to the relevant diagrams

*alertService: ExternalAlertSystem** was added to the *Notification* observer with a relating variable added to the Notification constructor

ExternalAlertSystem is now using composition with *Notification*

Version 4:



Revision 1 - Removal of the Decorator pattern:

The problem or question being considered

The functionality provided by the decorator pattern was not relevant to the workflows we implemented.

The original design or assumption

We originally had a generic workflow implementation and we thought that security, audit, validation and priority were needed.

The change that was made

Removal of the decorator classes

Why the change was made

When we finally decided on the two workflows our project would implement security, audit, validation and priority functionalities were not needed.

Updates to the relevant diagrams

Removal of *ItemDecorator*, *Security*, *Audit*, *Validation*, and *Priority*

Revision 2 - Removal of the Monitoring class in the Observer pattern:

The problem or question being considered

Why implement a monitoring observer when we already have notification, dependency and audit logger.

The original design or assumption

Our original idea was to have monitoring capabilities that would return an update about what was happening with the *WorkComponent* object

The change that was made

Removal of the Monitoring object

Why the change was made

Implementation of the notification, dependency and audit logger classes achieve the same functionality as the monitoring class

Updates to the relevant diagrams

Removal of the *Monitoring* class

Revision 3 - Changing name of DepthFirst and DepthFirst in Iterator Pattern:

The problem or question being considered

When the iterator pattern was implemented we were unsure what they were going to be used for so just adding the generic names just as placeholder for the different iterators.

The original design or assumption

Our original idea was to have monitoring capabilities that would return an update about what was happening with the *WorkComponent* object

The change that was made

Changed names of DepthFirst to DependencyIteraor and BreadthFirst to PriorityIterator

Why the change was made

Names were updated to be more descriptive for the purpose of the iterators

Updates to the relevant diagrams

Renaming *DepthFirst* class to *DependencyIteraor* class
Renaming *BreadthFirst* class to *PriorityIterator* class

Revision 4 - Removing SubProcess Class and renaming Stage to ItemGroup in Composite Pattern

The problem or question being considered

When the composite pattern was implemented we thought that work process items could be broken up into stages and sub-processes.

The original design or assumption

Our original idea was to have broken up stages and sub-processes which were both Composite implementation in the composite pattern

The change that was made

Removal of *SubProcess* Class and renaming *Stage*

Why the change was made

Subprocess and Stage were implemented to complete the same process. So the removal of subprocess helped to simplify the system.
The renaming of Stage to ItemGroup helps to imply that it is a group of item objects

Updates to the relevant diagrams

Rename *Stage* to *ItemGroup*
Removal of *SubProcess*

Revision 5 - Removing Memento Design Pattern

The problem or question being considered

When we originally designed our system we included the memento pattern so we would implement snapstores and restoration.

The original design or assumption

Our original design linked the command pattern and the memento pattern which implement snapshot taking and restoration.

The change that was made

Removal of the memento design pattern classes and functions from CommandInvoker

Why the change was made

When the new spec was release it was indicated that the memento pattern was not required so the removal helped to reduce the amount of patterns our system implemented. As well as receiving the complexity of our system

Updates to the relevant diagrams

Removal of the *WorkflowMemento* and *HistoryCaretaker* classes
Removal of the *rollback* attribute in *CommandInvoker*
Removal of the *redo()* function in *CommandInvoker*

Revision 6 - Adding Builder design pattern

The problem or question being considered

Using the builder design pattern was the best way to build and represent our two workflows of Aviation: Aircraft Maintenance Dispatch and Manufacturing: Production Order. With the added benefit of the inclusion of the builder pattern added a Creational pattern to our system.

The original design or assumption

When the original system was designed we did not have a plan on how to build our workflow implementations nor did we know about the benefits of the Builder pattern.

The change that was made

Additions of WorkflowBuilder and AviationWorkflowBuilder and ManufacturingWorkflowBuilder classes and added functions to Workflow Manager

Why the change was made

Adding these classes and functions implements the builder design pattern

Updates to the relevant diagrams

Addition of *WorkflowBuilder* and *AviationWorkflowBuilder* and *ManufacturingWorkflowBuilder* classes
Addition of function to *WorkflowManager* to make it a Director

Revision 7 - Adding Prototype design pattern

The problem or question being considered

Implementation of the prototype pattern helps to implement the cloning of *WorkComponent* objects

The original design or assumption

When the original design was made we did not think that cloning functionalities were needed for *WorkComponents*.

The change that was made

Additions of prototype design pattern functions into composite design pattern

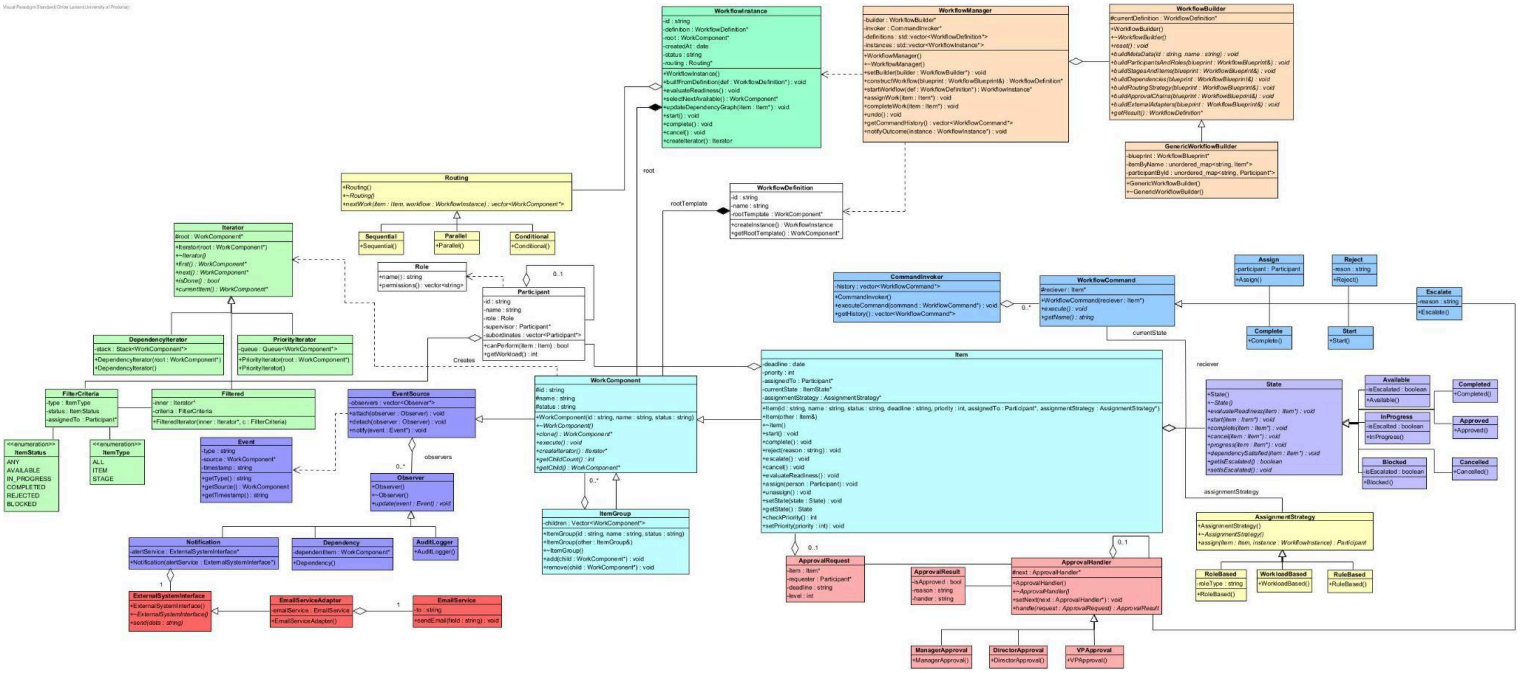
Why the change was made

Adding these functions implements the prototype design pattern

Updates to the relevant diagrams

Addition of pure virtual *clone()* function in *WorkComponent*
Addition of copy constructors in *ItemGroup* and *Item*

Version 5:



Revision 1 - Removal of the LegacyCRM in the Adapter pattern:

The problem or question being considered

The functionality provided by the LegacyCRM was also provided by the EmailService.

The original design or assumption

We originally had an email service for internal communication and the CRM to external communication to the clients.

The change that was made

Removal of LegacyCRM and it related adapter

Why the change was made

We decided that the email service could achieve the same purpose as the CRM so the removal of the CRm would help simplify the implementation of the adapter pattern.

Updates to the relevant diagrams

Removal of *LegacyCRMAdapter*, and *LegacyCRM*

Revision 2 - Change to Builder pattern

The problem or question being considered

The functionality provided by the builder pattern was too specific and it must be more general for better implementation

The original design or assumption

We originally designed the builder to build specifically for our two chosen workflows.

The change that was made

Update to WorkflowBuilder, removal of AviationWorkflowBuilder and ManufacturingWorkflowBuilder and addition of genericWorkflowBuilder

Why the change was made

But changing the concrete builders to be a generic build helps to make our system more versatile for different industries.

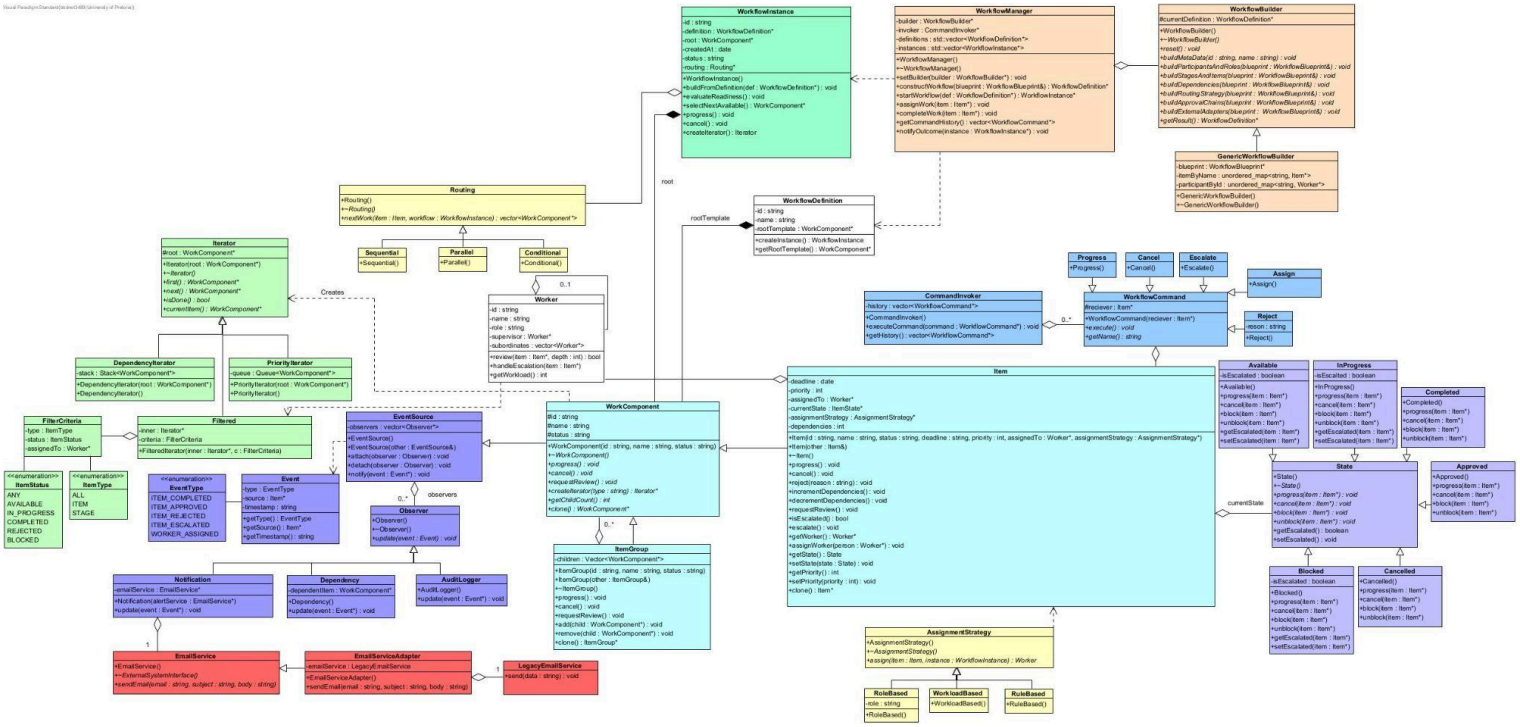
Updates to the relevant diagrams

Removal of *AviationWorkflowBuilder* and *ManufacturingWorkflowBuilder*

Adding *GenericWorkflowBuilder*

Updating functions in WorkflowBuilder to accept a WorkflowBlueprint object as a parameter

Version 6



Revision 1 - Simplification to Chain of Responsibility

The multi-class structure we had prior was unnecessarily bulky for the task of reviewing an Item in a chain of superiors. Therefore, the whole system was moved into a single method, review(), in Worker (previously Participant).

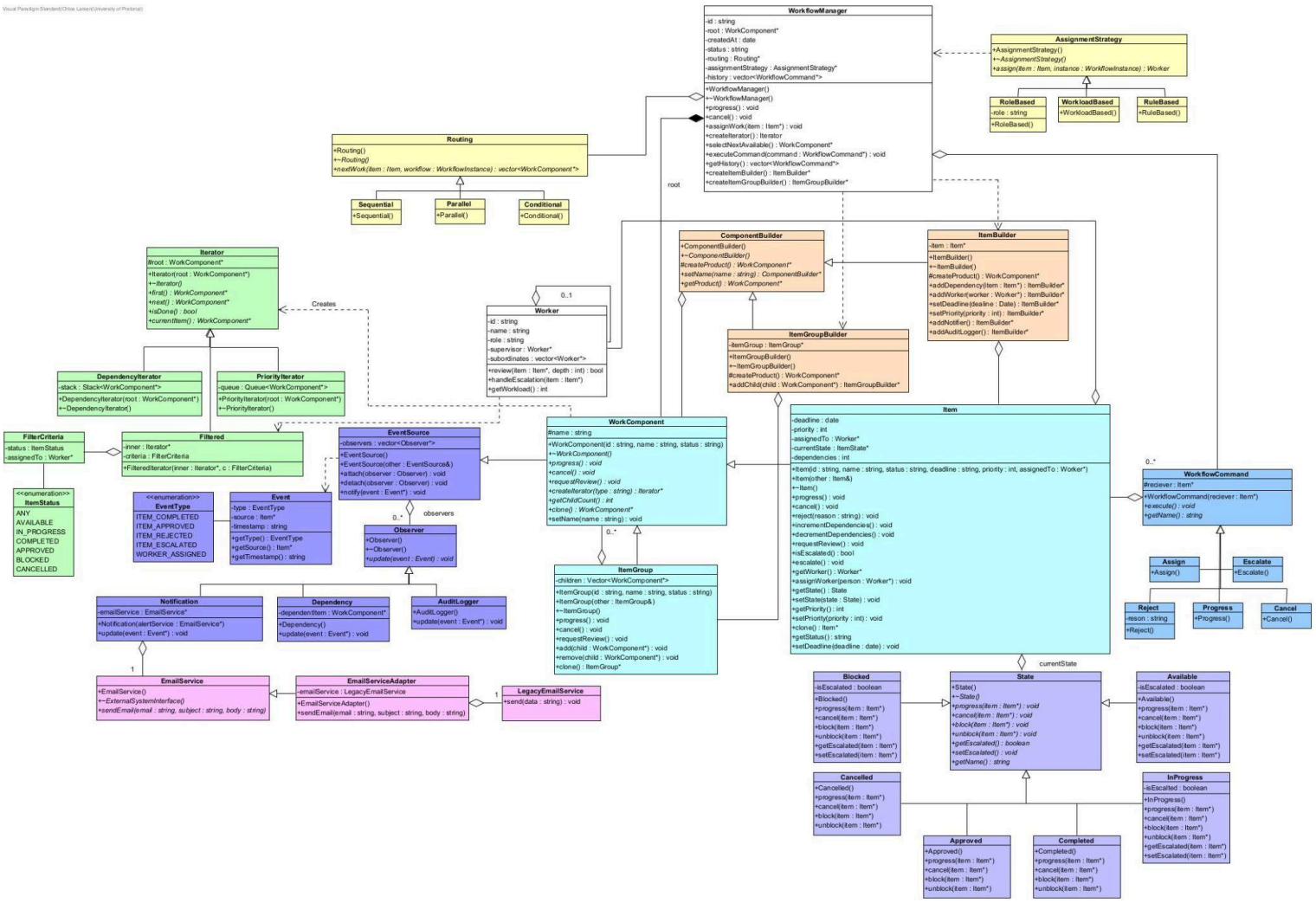
Revision 2 - Email System Update

Now, the legacy system accepts parameters in a deprecated way, taking in everything as a single string. So, the target interface is used for better readability by separating each bit into its own parameter.

Revision 3 - Composite & State Updates

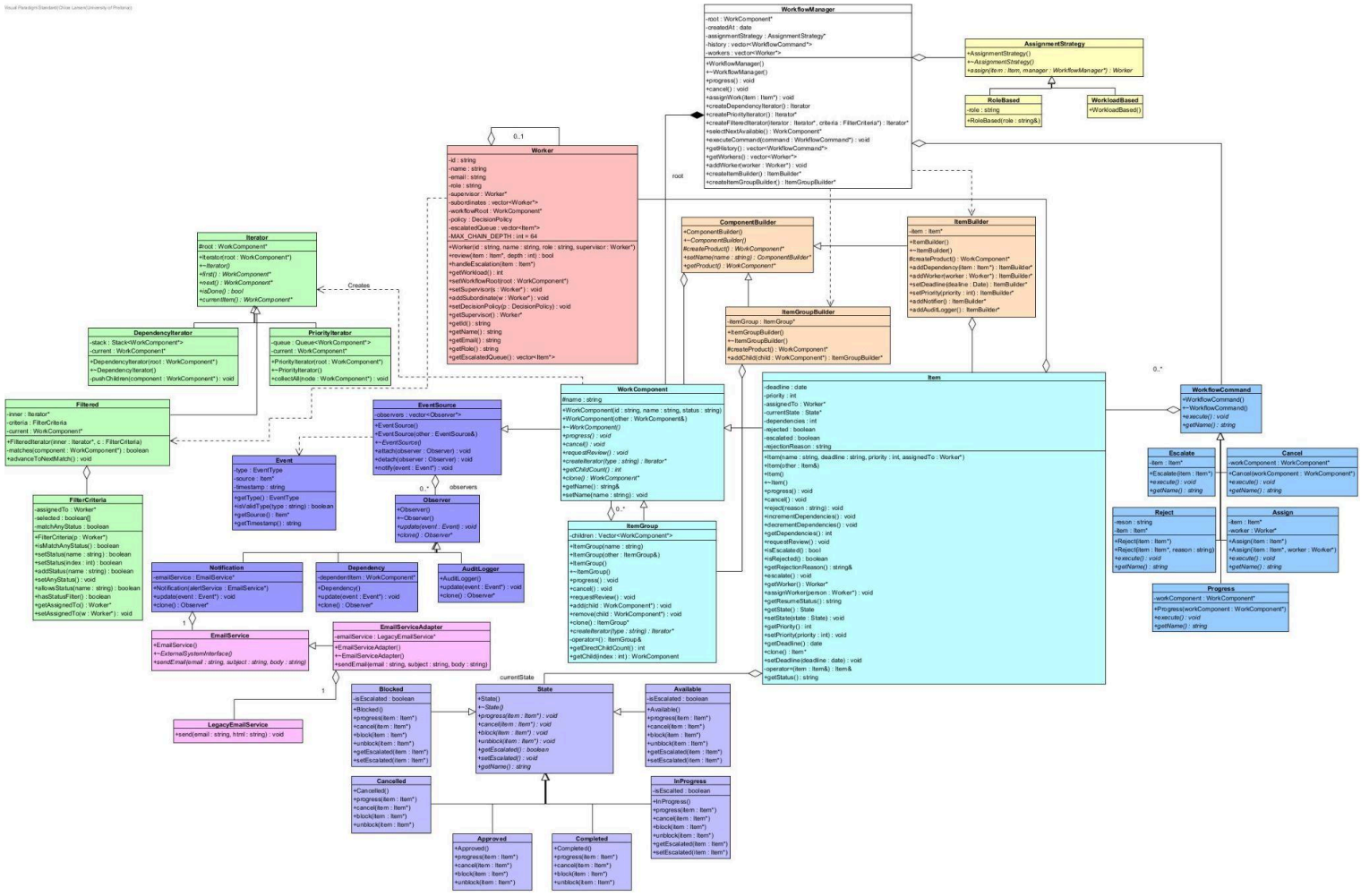
Added abstract methods to Component that better fit the composite design pattern by having the methods in ItemGroup forward the methods to each of its children. Similarly, State was updated to match those methods and better handle different tasks uniquely.

Version 7



- Simplified WorkflowManager, WorkflowInstance and WorkflowDefinition to just WorkflowManager (others were redundant)
- Merged CommandInvoker with WorkflowManager
- Redefined builder design pattern which uses factory methods for creating WorkComponents
- Updated WorkflowManager methods to be more consistent with the rest of the design
- Simplified Filtered iterator

Version 8



- Removing of routing in the strategy pattern
- Removing Rule based assignment strategy from the strategy pattern
- Updating most classes to ensure it matches the implemented code
- Removal of the enums and changes to classes where enums were used to rather use an array of strings
- Change email service so that it returns an html output.
- Changing assignment strategy arrow to composition

Task 8: GitHub Workflow

Team Contribution Statement

We used GitHub for collaboration as follows: Whenever one of us wanted to work on a new feature, we created a new branch locally, made a few commits with the new changes, and then pushed the branch to the repo. Once on the repo, the person who made those changes opened a pull request for the branch and requested review from one or more of the other members. If the changes were not approved, the pull request was closed and the person making the changes had to revise their additions. Otherwise, the changes are merged into main and the feature branch gets auto closed. We also used issues to keep track of who still had to do what, and sometimes attached issues to pull requests to automatically close the issue along with the pull request. We made use of rule sets to ensure that pull requests would only be merged when there has been one or more approved reviews. The used rule set also enforced that no push can be done to the main branch.

Branches and pull requests were very helpful because they let us review each other's code before merging it into main, meaning all of our code on the main branch is agreed to be quality code.

We divided the work based on functionality.
For practical 6:

- Anchen Kruger -> Task 3 - Activity Diagram 1, Task 6 - State diagram
- Caleb Jennings -> Task 3 - Activity Diagram 2, Task 6 - Sequence diagram 1, Task 8
- Caitlin Moodley -> Task 2, Task 6 - Sequence diagram 2
- Chloe Larsen -> Task 4, Task 5 and Task 7
- Shanna Reinecke -> Task 1, Task 3 - Activity Diagram 3

Project Implementation:

- Anchen Kruger -> State and Strategy pattern implementation and doxygen comments. Generating the doxygen fike
- Caleb Jennings -> Adapter and observer pattern implementation and doxygen comments. WorkflowManager implementation.
- Caitlin Moodley -> Composite and Command pattern implementation and doxygen comments. Implementing GUI.
- Chloe Larsen -> Factory Method and Iterator pattern implementation and doxygen comments. Finalising the class diagram
- Shanna Reinecke -> Prototype and Chain pattern implementation and doxygen comments. Created the demo presentation

Task 9: Build/run/test instructions

Build:

make -C gui -j4

Run:

./gui/build/tividy-gui

Test:

make test

```cpp
SECTION("Item progression works correctly")
{
    Item item;

    REQUIRE(item.getState() != nullptr);
    REQUIRE(item.getState()->getName() == "AVAILABLE");

    item.progress();

    REQUIRE(item.getState()->getName() == "IN_PROGRESS");

    item.progress();

    REQUIRE(item.getState()->getName() == "COMPLETED");
}
```

```
tests/item_tests.cpp:48: FAILED:
  REQUIRE( item2.getState()->getName() == "COMPLETEDA" )
with expansion:
  "COMPLETED" == "COMPLETEDA"

===============================================================================
test cases:  2 |  1 passed | 1 failed
assertions: 26 | 25 passed | 1 failed
```

```
===============================================================================
All tests passed (26 assertions in 2 test cases)
```